МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Н.Э. БАУМАНА
(национальный исследовательский университет)
Факультет «Информатика и системы управления» | Кафедра «Программная инженерия» (ИУ7)
Дисциплина: «Программирование и конструирование языков программирования» (ПиК ЯП)

ОТЧЕТ ПО РУБЕЖНОМУ КОНТРОЛЮ №1

Разработка программ на языке Python с использованием функционального стиля

Студент:	_____
Учебная группа:	_____
Вариант предметной области:	**№ 9 («Операционная система» — «Компьютер»)**
Вариант запросов:	**Вариант Б**

1. Описание классов данных предметной области

В соответствии с условиями рубежного контроля (вариант предметной области № 9) разработаны классы данных, представляющие сущности «Операционная система» (сторона отношения «многие») и «Компьютер» (сторона отношения «один»), а также ассоциативный класс для отношения «многие-ко-многим».

Класс	Поля класса	Назначение и связь
OS (Операционная система)	`id` (int) `name` (str) `price` (int/float) `comp_id` (int)	Сущность на стороне связи «многие» (аналог «Сотрудник»). Содержит количественный признак `price` (стоимость лицензии) и внешний ключ `comp_id` для связи один-ко-многим с компьютером.
Comp (Компьютер)	`id` (int) `name` (str)	Сущность на стороне связи «один» (аналог «Отдел»). Представляет собой рабочий компьютер или сервер организации.
OSComp (Связь ОС и Компьютеров)	`comp_id` (int) `os_id` (int)	Таблица/класс связки для реализации отношения «многие-ко-многим». Отражает установку нескольких ОС на различных компьютерах.

2. Постановка и адаптация запросов (Вариант Б)

Запросы из варианта Б перенесены в предметную область №9 с сохранением логических критериев и функциональных требований:

> **Запрос Б1 (один-ко-многим):** «Компьютер» и «Операционная система» связаны соотношением один-ко-многим. Выведите список всех связанных операционных систем и компьютеров, отсортированный по операционным системам (по названию ОС), сортировка по компьютерам произвольная.

> **Запрос Б2 (один-ко-многим):** «Компьютер» и «Операционная система» связаны соотношением один-ко-многим. Выведите список компьютеров с количеством операционных систем на каждом

> компьютере, отсортированный по количеству операционных систем (по убыванию).

> **Запрос Б3 (многие-ко-многим):** «Компьютер» и «Операционная система» связаны соотношением многие-ко-многим. Выведите список всех операционных систем, у которых название заканчивается на «Linux» (адаптация критерия варианта Б *«фамилия заканчивается на «ов»»* для предметной области с использованием метода `str.endswith`), и названия их компьютеров.

3. Исходный код программы (main.py)

```
# используется для сортировки
from operator import itemgetter


class OS:
    """Операционная система"""
    def __init__(self, id, name, price, comp_id):
        self.id = id
        self.name = name
        self.price = price
        self.comp_id = comp_id


class Comp:
    """Компьютер"""
    def __init__(self, id, name):
        self.id = id
        self.name = name


class OSComp:
    """
    'Операционные системы на компьютере' для реализации
    связи многие-ко-многим
    """
    def __init__(self, comp_id, os_id):
        self.comp_id = comp_id
        self.os_id = os_id


# Компьютеры
comps = [
    Comp(1, 'Сервер баз данных'),
    Comp(2, 'Рабочая станция разработчика'),
    Comp(3, 'Офисный ПК бухгалтерии'),
    Comp(4, 'Вычислительный кластер'),
    Comp(5, 'Тестовый стенд'),
]

# Операционные системы
oses = [
    OS(1, 'Ubuntu Linux', 0, 1),
    OS(2, 'Debian Linux', 0, 2),
    OS(3, 'Arch Linux', 0, 2),
    OS(4, 'Windows 11 Pro', 18000, 3),
    OS(5, 'Astra Linux', 12000, 3),
    OS(6, 'Red Hat Enterprise Linux', 25000, 3),
]

# Связь многие-ко-многим
os_comps = [
    OSComp(1, 1),
    OSComp(1, 2),
    OSComp(2, 2),
```

```
    OSComp(2, 3),
    OSComp(3, 4),
    OSComp(3, 5),
    OSComp(4, 1),
    OSComp(4, 6),
    OSComp(5, 5),
    OSComp(5, 6),
]


def main():
    """Основная функция"""

    # Соединение данных один-ко-многим
    one_to_many = [
        (os.name, os.price, comp.name)
        for comp in comps
        for os in oses
        if os.comp_id == comp.id
    ]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [
        (comp.name, oc.comp_id, oc.os_id)
        for comp in comps
        for oc in os_comps
        if comp.id == oc.comp_id
    ]

    many_to_many = [
        (os.name, os.price, comp_name)
        for comp_name, comp_id, os_id in many_to_many_temp
        for os in oses
        if os.id == os_id
    ]

    print('Задание Б1')
    # Сортировка по операционным системам (по названию ОС, первый элемент кортежа)
    res_1 = sorted(one_to_many, key=itemgetter(0))
    print(res_1)

    print('\nЗадание Б2')
    res_2_unsorted = []
    # Перебираем все компьютеры
    for c in comps:
        # Список ОС компьютера
        c_oses = list(filter(lambda i: i[2] == c.name, one_to_many))
        # Если на компьютере есть связанные ОС
        if len(c_oses) > 0:
            # Количество операционных систем на компьютере
            c_count = len(c_oses)
            res_2_unsorted.append((c.name, c_count))

    # Сортировка по количеству операционных систем (по убыванию)
    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    print(res_2)

    print('\nЗадание Б3')
    res_3 = {}
    # Перебираем все операционные системы
    # Адаптация условия "фамилия заканчивается на 'ов'": название ОС заканчивается на "Linux"
    for os in oses:
        if os.name.endswith('Linux'):
            # Список компьютеров для данной ОС
            os_comps_list = list(filter(lambda i: i[0] == os.name, many_to_many))
            # Только названия компьютеров
            comp_names = [comp_name for _, _, comp_name in os_comps_list]
```

```
            # Добавляем в словарь: ключ - ОС, значение - список компьютеров
            res_3[os.name] = comp_names
    print(res_3)


if __name__ == '__main__':
    main()
```

4. Результаты выполнения программы

Результаты выполнения программы, полученные при запуске интерпретатором Python:

```
Задание Б1
[('Arch Linux', 0, 'Рабочая станция разработчика'), ('Astra Linux', 12000, 'Офисный ПК
бухгалтерии'), ('Debian Linux', 0, 'Рабочая станция разработчика'), ('Red Hat Enterprise Linux',
25000, 'Офисный ПК бухгалтерии'), ('Ubuntu Linux', 0, 'Сервер баз данных'), ('Windows 11 Pro',
18000, 'Офисный ПК бухгалтерии')]

Задание Б2
[('Офисный ПК бухгалтерии', 3), ('Рабочая станция разработчика', 2), ('Сервер баз данных', 1)]

Задание Б3
{'Ubuntu Linux': ['Сервер баз данных', 'Вычислительный кластер'], 'Debian Linux': ['Сервер баз
данных', 'Рабочая станция разработчика'], 'Arch Linux': ['Рабочая станция разработчика'], 'Astra
Linux': ['Офисный ПК бухгалтерии', 'Тестовый стенд'], 'Red Hat Enterprise Linux': ['Вычислительный
кластер', 'Тестовый стенд']}
```

5. Краткие выводы и пояснения

При выполнении рубежного контроля решены все поставленные задачи:

1. Созданы классы данных `OS`, `Comp` и `OSComp` в строгом соответствии с предметной областью №9 («Операционная система» — «Компьютер»). В класс `OS` включен количественный признак `price` (стоимость лицензии), а также внешний ключ `comp_id`.
2. Сформированы тестовые наборы данных, содержащие 5 компьютеров, 6 операционных систем и 10 связей многие-ко-многим, гарантирующие полноту и корректность проверки всех трех запросов.
3. Для формирования связей один-ко-многим и многие-ко-многим применены списковые включения (*list comprehensions*), моделирующие реляционные операции соединения таблиц.
4. В запросах активно использованы функциональные возможности Python:
    - встроенная функция `sorted()` с извлечением ключа через `operator.itemgetter`;
    - функция высшего порядка `filter()` в связке с анонимными функциями `lambda`;
    - метод `str.endswith()` для эффективной строковой фильтрации.